



ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ

Τμήμα Μηχανικών Πληροφορικής και Υπολογιστών

Μάθημα: Υπολογιστική Όραση

Απαλλακτική Εργασία

Θέμα:

Ανίχνευση, Παρακολούθηση Παικτών σε Πραγματικό Χρόνο και Αυτόματη Εξαγωγή Αθλητικών Στατιστικών Ποδοσφαίρου (Real-Time Football Analytics: Player Tracking using YOLO and ByteTrack)

Ερευνητικά Ερωτήματα:

- Πώς μπορεί να επιτευχθεί σταθερή παρακολούθηση πολλαπλών αντικειμένων (Multi-Object Tracking) σε συνθήκες γρήγορης κίνησης, έντονων εναλλαγών κάμερας και αλληλοεπικάλυψης (occlusions) μεταξύ των παικτών;
- Πώς μπορεί το σύστημα να ξεχωρίσει αυτόματα τις δύο ομάδες χωρίς χειροκίνητη παρέμβαση, βασιζόμενο αποκλειστικά στα οπτικά χαρακτηριστικά των εμφανίσεων (jersey color classification);
- Με ποιο τρόπο μπορούν οι συντεταγμένες των παικτών να μετατραπούν σε χρήσιμα οπτικά στατιστικά, όπως heatmaps κίνησης;

Φοιτητής: Θεμιστοκλής Μαρτσούκος

ΑΜ: 20390146

Ακαδημαϊκό Έτος: 2025–2026

ΚΕΦΑΛΑΙΟ 1: ΕΙΣΑΓΩΓΗ ΚΑΙ ΟΡΙΣΜΟΣ ΠΡΟΒΛΗΜΑΤΟΣ

Η αυτοματοποιημένη εξαγωγή τακτικών δεδομένων από τηλεοπτικές μεταδόσεις (broadcast footage) αποτελεί κομβικό πεδίο έρευνας στα Sports Analytics. Η παραδοσιακή, χειροκίνητη καταγραφή στατιστικών (όπως η χωρική κατανομή παικτών και τα ποσοστά κατοχής) είναι χρονοβόρα και επιρρεπής σε υποκειμενικά σφάλματα. Η παρούσα εργασία επιλύει αυτό το πρόβλημα αναπτύσσοντας ένα αυτοματοποιημένο, modular pipeline υπολογιστικής όρασης για την επεξεργασία ποδοσφαιρικών αγώνων σε πραγματικό χρόνο.

Το σύστημα αντιμετωπίζει τρία κεντρικά ερευνητικά ερωτήματα: (α) τη σταθερή παρακολούθηση πολλαπλών αντικειμένων (Multi-Object Tracking) υπό συνθήκες γρήγορης κίνησης και αλληλοεπικαλύψεων (occlusions), (β) τον αυτόματο, μη επιβλεπόμενο διαχωρισμό των ομάδων (Unsupervised Clustering) βάσει των χρωματικών χαρακτηριστικών της εμφάνισης, και (γ) τη μετατροπή των χωρικών συντεταγμένων σε χρήσιμα οπτικά στατιστικά (Team-Specific Heatmaps). Για την επίτευξη μέγιστης ακρίβειας, το project επεκτείνει τις deep learning μεθοδολογίες ενσωματώνοντας έναν πρωτότυπο, υβριδικό μηχανισμό κλασικής υπολογιστικής όρασης βασισμένο στον μετασχηματισμό Hough, γεφυρώνοντας τα κενά ανίχνευσης μικρών αντικειμένων.

ΚΕΦΑΛΑΙΟ 2: ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΑΝΙΧΝΕΥΣΗΣ ΚΑΙ ΠΑΡΑΚΟΛΟΥΘΗΣΗΣ ΑΝΤΙΚΕΙΜΕΝΩΝ

2.1 Στατική Ανίχνευση Αντικειμένων (Object Detection - YOLOv8)

Η απομόνωση των οντοτήτων υλοποιείται στο υποσύστημα `src/detector.py` μέσω της κλάσης `FootballDetector`. Χρησιμοποιείται το συνελικτικό νευρωνικό δίκτυο `YOLOv8m` (`yolov8m.pt`), το οποίο εξασφαλίζει τη βέλτιστη ισορροπία μεταξύ χρόνου εκτέλεσης (inference time) και ακρίβειας. Η μέθοδος `detect()` δέχεται κάθε frame ως πίνακα BGR και εκτελεί μια stateless πρόβλεψη, εφαρμόζοντας φίλτρο κλάσεων για τη διατήρηση μόνο της κλάσης 0 (person - παίκτες, διαιτητές) και της κλάσης 32 (sports ball - μπάλα) από το COCO dataset. Κάθε ανίχνευση δομείται σε λεξικό με τις συντεταγμένες πλαισίου, το σκορ εμπιστοσύνης (confidence) και το `class_id`. Για λόγους υπολογιστικής απόδοσης, το μοντέλο φορτώνεται μία φορά στην `init`, αποτρέποντας τη χρονοβόρα επαναφόρτωση βαρών ανά frame.

2.2 Δυναμική Παρακολούθηση Πολλαπλών Αντικειμένων (Multi-Object Tracking - ByteTrack)

Το υποσύστημα `src/tracker.py` (FootballTracker) μετατρέπει τις stateless ανιχνεύσεις σε συνεχείς τροχιές με τον αλγόριθμο ByteTrack. Μέσω της μεθόδου `track_frame()`, καλείται το API της Ultralytics με την παράμετρο `persist=True` και το αρχείο ρυθμίσεων `bytetrack.yaml`, διασφαλίζοντας τη διατήρηση των Track IDs μεταξύ των frames. Το αντικείμενο του μοντέλου YOLO μεταβιβάζεται ως όρισμα από το κεντρικό script διαχείρισης (`main.py`), τηρώντας την αρχή της ενιαίας ευθύνης. Ο ByteTrack λειτουργεί σε δύο μαθηματικά στάδια:

1. **Φίλτρο Kalman:** Αναλύει το διάνυσμα θέσης και ταχύτητας κάθε οντότητας, προβλέποντας τη γεωμετρική της θέση στο επόμενο frame και γεφυρώνοντας τις στιγμιαίες αλληλοεπικαλύψεις (occlusions).
2. **Αλγόριθμος Hungarian:** Επιλύει το πρόβλημα της βέλτιστης γραμμικής αντιστοίχισης (Linear Assignment Problem), ελαχιστοποιώντας το συνολικό κόστος της Ευκλείδειας μετατόπισης μεταξύ των προβλέψεων και των νέων ανιχνεύσεων. Σε περίπτωση απώλειας ID της μπάλας, ο κώδικας εκχωρεί μια τιμή αναμονής (`track_id = -1`) για την αποφυγή σφαλμάτων κατάρρευσης.

2.3 Υβριδικός Μηχανισμός Τριών Επιπέδων και Fallback Ανίχνευσης Μπάλας (HoughCircles)

Σε πραγματικό τηλεοπτικό βίντεο ανάλυσης 1920*1080pixels (Paris FC vs PSG), η ανάκληση (recall) της μπάλας ήταν αρχικά εξαιρετικά χαμηλή (~1–5%) λόγω ασυμβατότητας κλίμακας (καταλαμβάνει μόλις το 0.02%-0.05% του frame), έντονου θολώματος κίνησης (motion blur ~55 px/frame στα 30 fps) και χάσματος τομέα (domain gap) μεταξύ COCO και broadcast πλάνων. Για την επίλυση αυτού του δομικού περιορισμού, σχεδιάστηκε και υλοποιήθηκε μια πρωτότυπη ιεραρχία ανίχνευσης τριών επιπέδων (3-Tier Cascade Chain) στο `main.py`:

- **Tier 1 (ByteTrack):** Το σύστημα ελέγχει αν ο stateful tracker έχει ήδη ενεργό και επίμονο `track_id` για την μπάλα από προηγούμενα καρέ.
- **Tier 2 (YOLO Fallback):** Αν το Tier 1 αποτύχει, ενεργοποιείται η μέθοδος `detect_ball()` με διαφοροποιημένο, χαμηλό κατώφλι εμπιστοσύνης (`conf=0.10`) για να ανασύρει οριακές deep learning ανιχνεύσεις.
- **Tier 3 (BallHoughDetector):** Αν αποτύχουν και τα δύο πρώτα επίπεδα, επιστρατεύεται το νέο module `src/ball_hough.py`.

Η κλάση `BallHoughDetector` βασίζεται σε κλασικές μεθόδους επεξεργασίας εικόνας, εκτελώντας διαδοχικά βήματα: μετατροπή σε γκρι κλίμακα (`cv2.cvtColor`), απαλοιφή θορύβου μέσω Γαουσιανού φίλτρου (`cv2.GaussianBlur` με πυρήνα 9*9) και εφαρμογή του κυκλικού μετασχηματισμού Hough (`cv2.HoughCircles`). Στη συνέχεια, εφαρμόζεται χωρικό φιλτράρισμα ROI εντός του γηπέδου και ένα Φίλτρο Φωτεινότητας (Brightness

Heuristic), το οποίο ελέγχει αν η μέση τιμή των pixels του υποψήφιου κύκλου είναι $\text{mean_brightness} > 155$. Η συγκεκριμένη επιλογή αποδείχθηκε κομβική, καθώς αποκλείει αποτελεσματικά τα false positives που προκαλούνται από τα κεφάλια των παικτών (τα οποία λόγω tactical λήψης μοιάζουν με κύκλους), χωρίς την ανάγκη κάποιου βαριού ML μοντέλου. Η εισαγωγή του Tier 3 μπορεί να φτάσει το ball detection rate στο 35–50% των frames με χαλαρές παραμέτρους. Ωστόσο, για την τελική δοκιμή επιλέχθηκε μια στρατηγική αυστηρού γεωμετρικού φιλτραρίσματος ($\text{MAX_BALL_RADIUS} = 6$, $\text{HOUGH_ACCUMULATOR_THRESHOLD} = 26$). Αυτό περιόρισε την ανάκληση (Recall) στο 1.0%, αλλά εξασφάλισε 100% ακρίβεια (Precision), εκμηδενίζοντας πλήρως τα false positives από τις λευκές κάλτσες των παικτών και διασφαλίζοντας την εγκυρότητα των τελικών στατιστικών

ΚΕΦΑΛΑΙΟ 3: ΜΗ ΕΠΙΒΛΕΠΟΜΕΝΗ ΜΗΧΑΝΙΚΗ ΜΑΘΗΣΗ ΚΑΙ ΔΙΑΧΩΡΙΣΜΟΣ ΟΜΑΔΩΝ

3.1 Απομόνωση και Προεπεξεργασία Περιοχής Ενδιαφέροντος (Torso Cropping & HSV Grass Masking)

Ο αυτόματος διαχωρισμός των ομάδων εκτελείται στο υποσύστημα `src/classifier.py` μέσω της κλάσης `TeamClassifier`. Αρχικά, απομονώνεται γεωμετρικά ο άνω κορμός του παίκτη (torso crop) στο ύψος μεταξύ 20% και 50% του bounding box, και οριζόντια στο κεντρικό 60% του πλάτους, παρακάμπτοντας το κεφάλι, τα σορτσάκια και το background bleed. Λόγω της tactical λήψης, το πράσινο γρασίδι κυριαρχούσε στα crops (Grass Bleeding), αναγκάζοντας τον K-Means να συγκλίνει εσφαλμένα σε κέντρα με τιμές Hue (H 45-55). Το πρόβλημα λύθηκε με την εισαγωγή δυαδικής μάσκας μέσω της `cv2.inRange` στον χρωματικό χώρο HSV, ο οποίος προσφέρει σταθερότητα έναντι μεταβολών φωτισμού και σκιών. Ορίστηκαν τα όρια $\text{GRASS_LOWER} = [35, 40, 40]$ και $\text{GRASS_UPPER} = [75, 255, 255]$ (H = 35-75 στην OpenCV). Η μάσκα αντιστράφηκε με την `cv2.bitwise_not` και ο υπολογισμός της μέσης τιμής εκτελέστηκε αποκλειστικά για τα pixels της φανέλας μέσω της `cv2.mean(src, mask=jersey_mask)`.

3.2 Σχεδιασμός Χώρου Χαρακτηριστικών για Αχρωματικές Εμφανίσεις (Feature Space Redesign)

Στην αναμέτρηση Paris FC (λευκά) εναντίον PSG (σκούρα μπλε), ο αρχικός τρισδιάστατος χώρος [H, S, V] απέτυχε, καθώς οι αχρωματικές (near-achromatic) εμφανίσεις παρουσιάζουν χαμηλό κορεσμό, με αποτέλεσμα η τιμή Hue να συμπεριφέρεται ως καθαρός θόρυβος. Για την υπέρβαση του περιορισμού, ο χώρος χαρακτηριστικών μειώθηκε σε ένα δισδιάστατο διάνυσμα [S, V], βασιζόμενο αποκλειστικά στον Κορεσμό (Saturation) και τη Φωτεινότητα (Value). Η φωτεινότητα αποτέλεσε τον στατιστικά αξιόπιστο άξονα διαχωρισμού. Ο K-Means εντόπισε με ακρίβεια τα δύο χρωματικά κέντρα: το Cluster 0 κλείδωσε σε $S = 60.2$, $V = 179.3$ (λευκά

Paris FC) και το Cluster 1 σε $S = 139.4$, $V = 82.4$ (σκούρα PSG), παρέχοντας ένα γραμμικά διαχωρίσιμο όριο απόφασης με γεωμετρικό χάσμα 97 μονάδων στον άξονα V.

3.3 Μαθηματική Μοντελοποίηση K-Means και Φιλτράρισμα Ακραίων Τιμών (Outlier Pre-filtering)

Η ομαδοποίηση βασίζεται στον αλγόριθμο K-Means με παραμέτρους $k=2$, $n_init=10$ (για αποφυγή τοπικών ελαχίστων) και σταθερό $random_state=42$. Η εκτέλεση χωρίζεται σε δύο φάσεις στο `main.py`: (α) Φάση Εκπαίδευσης (Frames 0–29): Συλλογή 685 δειγμάτων παικτών και εκτέλεση `kmeans.fit()`, και (β) Φάση Πρόβλεψης (Frames 30+): Real-time ταξινόμηση κάθε νέου bounding box μέσω της `predict_team()`.

Για την αποφυγή παρεμβολής του διαιτητή με τη neon φανέλα (Referee Cluster Hijacking), ο οποίος λόγω ακραίου κορεσμού ($S = 220-255$) «έκλεβε» το ένα από τα δύο clusters, υλοποιήθηκε προ-φίλτρο ακραίων τιμών πριν από τη συνάρτηση `kmeans.fit()`. Τέθηκε ανώτατο όριο κορεσμού εκπαίδευσης $MAX_SATURATION_FOR_FIT = 160$ και ελάχιστο όριο pixels $MIN_JERSEY_PIXELS = 50$. Τα δείγματα που υπερβαίνουν τα όρια απορρίπτονται κατά την εκπαίδευση, διασφαλίζοντας ότι το μοντέλο θα κλειδώσει καθαρά πάνω στις δύο ομάδες. Στο inference, ο διαιτητής κατατάσσεται στο πλησιέστερο cluster χωρίς να αλλοιώνει τη μαθηματική δομή των ομάδων.

ΚΕΦΑΛΑΙΟ 4: ΧΩΡΙΚΗ ΑΝΑΛΥΣΗ ΚΑΙ ΠΑΡΑΓΩΓΗ ΤΑΚΤΙΚΩΝ ΣΤΑΤΙΣΤΙΚΩΝ

Το τέταρτο στάδιο, υλοποιημένο στο `visualizer.py` και ορχηστρωμένο από το `main.py`, μετατρέπει τη ροή ανιχνεύσεων σε τακτικά στατιστικά. Πρώτο πρόβλημα είναι ο χωρικός θόρυβος: το YOLOv8 εντοπίζει και άτομα εκτός γηπέδου (προπονητές, αναπληρωματικοί), που είναι σχεδόν στάσιμοι και δημιουργούν ψευδή hot-spots στις άκρες. Η λύση είναι ένα πολύγωνο ROI που ορίζει τον ενεργό αγωνιστικό χώρο και προ-υπολογίζεται κατά την κατασκευή του FootballVisualizer μέσω `cv2.fillPoly()`. Η τιμή `bottom_roi_y = height - 15` κάνει το ROI δυναμικά ανεξάρτητο ανάλυσης, λειτουργώντας τόσο για 720p όσο και για 1080p. Κάθε έλεγχος εντός-ROI κοστίζει $O(1)$ - μία ανάγνωση πίνακα `roi_mask[foot_y, foot_x]` - αντί για χρονοβόρο κλήση `cv2.pointPolygonTest()`.

Για τη χωρική αποτύπωση χρησιμοποιείται το ground-contact point (κάτω-κεντρικό σημείο bounding box, `foot_y = y2`) αντί του κέντρου, ώστε η θέση του παίκτη να αντιστοιχεί στο επίπεδο του γηπέδου. Ο θερμικός χάρτης δομείται ως 2D accumulator float32 ανά ομάδα: κάθε καρέ αυξάνει κατά 1.0 την τιμή του αντίστοιχου pixel. Για τη μετατροπή του αραιού πίνακα σε ομαλή επιφάνεια πυκνότητας, εφαρμόζεται GaussianBlur με kernel 151×151 , ισοδύναμο με Kernel Density Estimation (KDE) με Gaussian kernel. Στη συνέχεια, κανονικοποίηση $NORM_MINMAX$ στο $[0, 255]$ και χρωματισμός `COLORMAP\_JET` αποδίδουν το τελικό αποτέλεσμα. Δύο ξεχωριστοί χάρτες - ένας ανά ομάδα - επιτρέπουν ανεξάρτητη τακτική ανάλυση: αμυντικό σχήμα, ζώνες επίθεσης, εδαφική κυριαρχία.

Η μετρική κατοχής υλοποιεί Nearest-Neighbour Association: σε κάθε καρέ όπου η μπάλα είναι ορατή (χάρη και στην ενίσχυση του Hough fallback), υπολογίζεται η Ευκλείδεια απόσταση $\text{math.hypot}(dx, dy)$ μεταξύ κέντρου μπάλας και ποδιών κάθε παίκτη. Ο πλησιέστερος παίκτης εντός 80 pixels αποδίδεται ως κάτοχος — το κατώφλι αντιστοιχεί εμπειρικά σε ~0.5 μέτρα αγωνιστικής απόστασης. Τέλος, η τροχιά της μπάλας οπτικοποιείται με sliding-window deque μέγιστου μήκους 15 θέσεων, συνδέοντας τα σημεία με τμήματα αυξανόμενου πάχους (1–4 pixels) για εφέ εξάχνωσης.

ΚΕΦΑΛΑΙΟ 5: ΠΕΙΡΑΜΑΤΙΚΑ ΑΠΟΤΕΛΕΣΜΑΤΑ, ΠΕΡΙΟΡΙΣΜΟΙ ΚΑΙ ΣΥΜΠΕΡΑΣΜΑΤΑ

Το σύστημα έτρεξε χωρίς προβλήματα σε όλο το βίντεο του αγώνα. Στην πράξη, το `main.py` συντονίζει τα modules με μια απλή, διαδοχική σειρά για κάθε καρέ: πρώτα γίνεται το tracking των παικτών (`track_frame()`), μετά εκτελείται η ιεραρχία ανίχνευσης της μπάλας (`detect_ball()` YOLO fallback και, αν χρειαστεί, ο μετασχηματισμός Hough), στη συνέχεια αναγνωρίζεται η ομάδα του κάθε παίκτη (`predict_team()`) και στο τέλος ο visualizer αναλαμβάνει να σχεδιάσει τα οπτικά αποτελέσματα στην οθόνη (bounding boxes και τροχιά μπάλας) και να ενημερώσει το heatmap. Για λόγους βελτιστοποίησης και εξοικονόμησης πόρων CPU, τα στατιστικά της κατοχής υπολογίζονται στο υπόβαθρο ανά frame και εκτυπώνονται συγκεντρωτικά στο τερματικό μόνο κατά την έξοδο από το πρόγραμμα. Μια πολύ σημαντική σχεδιαστική επιλογή είναι ότι η ταξινόμηση της ομάδας γίνεται μόνο μία φορά ανά καρέ για κάθε παίκτη. Το αποτέλεσμα αποθηκεύεται απευθείας στο layout του αντικείμενου ως `obj["team"]` και μοιράζεται σε όλες τις υπόλοιπες λειτουργίες. Αυτό μας γλιτώνει από περιττούς υπολογισμούς και κρατάει τον κώδικα καθαρό. Όταν το βίντεο ολοκληρωθεί, το σύστημα μας δίνει δύο εικόνες heatmap (μία για κάθε ομάδα) και τυπώνει στην κονσόλα τα τελικά ποσοστά της κατοχής.

Φυσικά, το pipeline έχει και κάποιους ξεκάθαρους περιορισμούς, με κυριότερο το ότι χρησιμοποιήσαμε το γενικό μοντέλο YOLOv8m εκπαιδευμένο στο COCO dataset, αντί για κάποιο εξειδικευμένο δίκτυο για ποδόσφαιρο. Τα anchor boxes του YOLO είναι φτιαγμένα για κανονικά αντικείμενα και όχι για μια μπάλα που πιάνει μόλις το 0.02% με 0.05% της οθόνης. Φυσικά, το pipeline έχει και κάποιους ξεκάθαρους περιορισμούς, με κυριότερο το ότι χρησιμοποιήσαμε το γενικό μοντέλο YOLOv8m εκπαιδευμένο στο COCO dataset, αντί για κάποιο εξειδικευμένο δίκτυο για ποδόσφαιρο. Τα anchor boxes του YOLO είναι φτιαγμένα για κανονικά αντικείμενα και όχι για μια μπάλα που πιάνει μόλις το 0.02% με 0.05% της οθόνης. Παρά τη δυνατότητα του BallHoughDetector να προσεγγίσει ένα ball recall της τάξης του 35-50% με χαλαρότερες παραμέτρους, στην τελική δοκιμή επιλέχθηκε η αυστηρή παραμετροποίησή του, σταθεροποιώντας το recall στο 1.0%. Αυτή η θυσία στην ανάκληση έγινε συνειδητά προκειμένου να εξασφαλιστεί 100% ακρίβεια (Precision), εξαλείφοντας πλήρως τα false positives από τις λευκές κάλτσες των παικτών της Paris FC. Παρόλα αυτά, οι κλασικές ευριστικές

μέθοδοι Hough έχουν περιορισμούς, καθώς επηρεάζονται από απότομες μεταβολές του φωτισμού ή έντονο motion blur κατά τη διάρκεια δυνατών σουτ. Αν γινόταν fine-tuning σε ένα ειδικό dataset όπως το SoccerNet, το recall θα μπορούσε να ξεπεράσει σταθερά το 90%.

Ένα άλλο ζήτημα είναι ότι ο υπολογισμός των αποστάσεων σε pixels δεν λαμβάνει υπόψη την προοπτική της κάμερας, με αποτέλεσμα οι παίκτες που είναι μακριά στο βάθος να φαίνονται τεχνητά πιο κοντά μεταξύ τους. Ως μελλοντική επέκταση, η καλύτερη λύση είναι το Homography Estimation. Με τη χρήση μιας προβολικής μήτρας 3×3 , θα μπορούσαμε να μετατρέψουμε κάθε pixel του βίντεο σε πραγματικά μέτρα πάνω στο χορτάρι. Αυτό θα επέτρεπε απόλυτα ακριβείς μετρήσεις ταχύτητας και τη σχεδίαση ενός heatmap κατευθειάν πάνω σε ένα επίσημο σχεδιάγραμμα γηπέδου της FIFA. Συνολικά, το project δείχνει ότι η ανάλυση ενός αγώνα είναι εφικτή ακόμα και με γενικά εργαλεία όρασης, αποτελώντας μια πολύ στέρεη βάση για μελλοντικές βελτιώσεις.

ΚΕΦΑΛΑΙΟ 6: Βιβλιογραφική Αναφορά

- Giancola, S., Amine, M., Dghaily, T., & Ghanem, B. (2018). SoccerNet: A scalable dataset for action spotting in soccer videos. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*.
- Varghese, R., & Sambath, M. (2024). YOLOv8: A Novel Object Detection Algorithm with Enhanced Performance and Robustness. *International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS)*.
- Crang, Z. L., et al. (2025). Concurrent validity of computer-vision artificial intelligence player tracking software using broadcast footage. *Journal of Sports Analytics*, 11(1), 45-58.
- Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., & Wang, X. (2022). ByteTrack: Multi-object tracking by associating every detection box. In *Proceedings of the European Conference on Computer Vision (ECCV)* (pp. 1-21).

